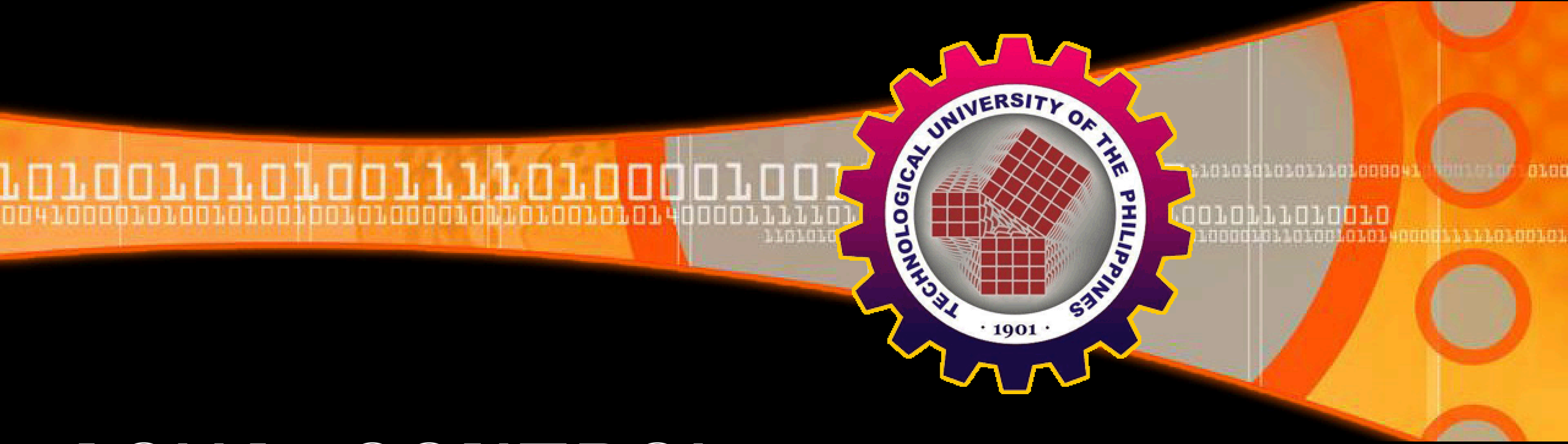


AQUA CONTROL

Automated Aquarium Water Quality Control System

Prepared By:
Flint Axl Celetaria
Jett Axel Manalo
Emma Rose Pascua

DESCRIPTION

This project focuses on an automated aquarium water quality control system that maintains optimal conditions for aquatic life. The system integrates multiple sensors and actuators to regulate temperature, turbidity, and pH levels efficiently.

1. **Temperature Control**
   - A DS18B20 waterproof temperature sensor continuously monitors the water temperature.
   - When the temperature drops below the desired threshold, a submersible aquarium heater(25W) is activated via a relay module.
   - The Arduino microcontroller ensures temperature stability by toggling the heater ON/OFF based on real-time readings.

DESCRIPTION

2. Turbidity Control

- A SKU SEN0189 turbidity sensor detects water clarity levels.
- When turbidity exceeds the set limit, a DC12V water pump is activated to initiate filtration and improve water clarity.

3. pH Stabilization

- A PH-4502C Liquid pH Sensor measures the water's pH level.
- If the pH deviates from the optimal range, a G328 peristaltic pump adds a stabilizing solution (acid or base) to correct the pH.
- The relay module controls the peristaltic pump to ensure accurate dosing.

Components



DS18B20
Temp Senor



Turbidity Sensor



PH-4502C Liquid pH Sensor



G328 Peristaltic Pump



Submersible Aquarium Heater



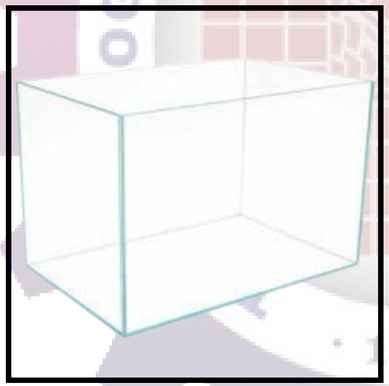
Dc12V Water Pump



Relay Module



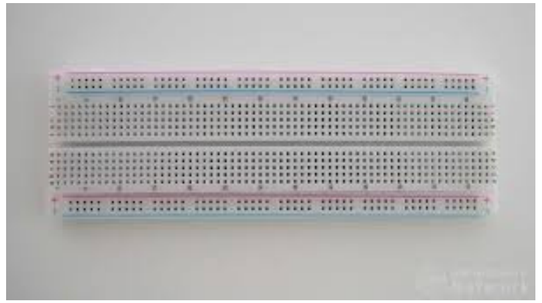
AC-DC Transformer 12v



3.5 gal aquarium



ESP32



BreadBoard

MAIN COMPONENTS

Microcontroller

- ESP32 – Handles sensor readings, actuator control, and Wi-Fi connectivity for remote monitoring.

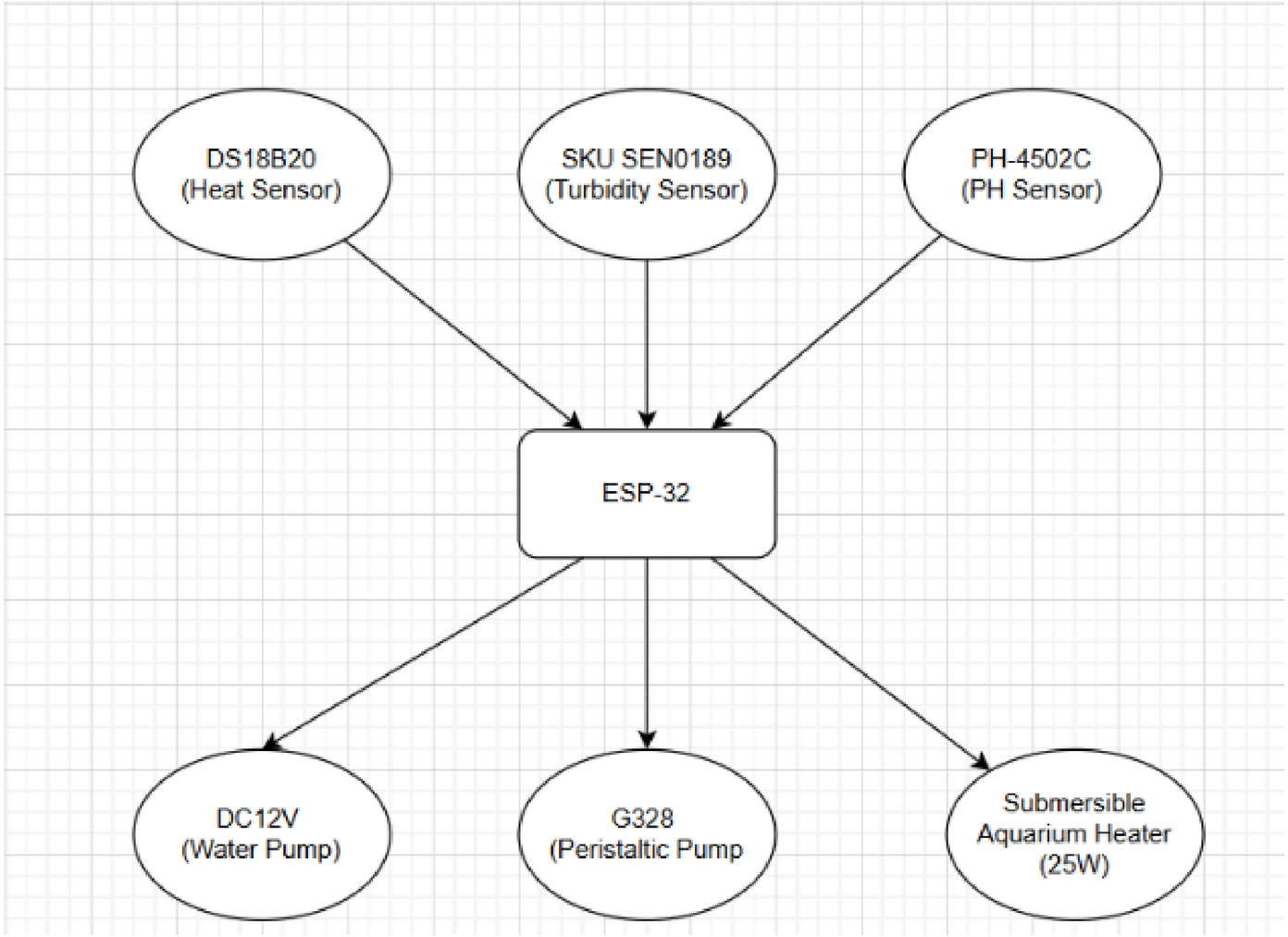
Inputs (Sensors)

- DS18B20 – Waterproof Temperature Sensor
- SKU SEN0189 – Turbidity Sensor
- PH-4502C Liquid pH Sensor – pH Monitoring

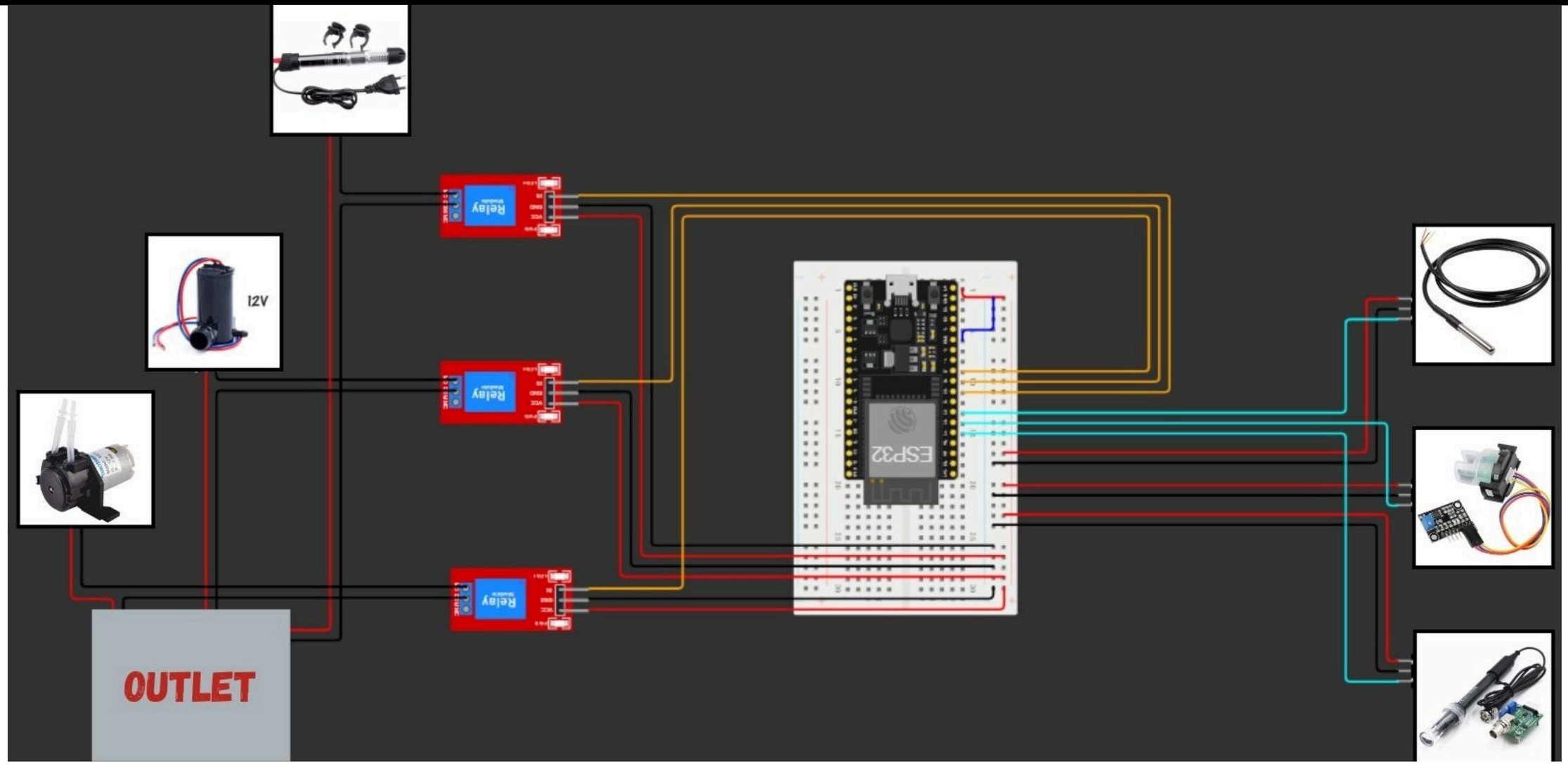
O**utputs**

- DC12V Water Pump – For turbidity control
- G328 Peristaltic Pump – For pH adjustment
- Submersible Aquarium Heater (25W)– For temperature regulation
- SRD-05VDC-SL-C Relay Module – Controls heater, DC pump and peristaltic pump

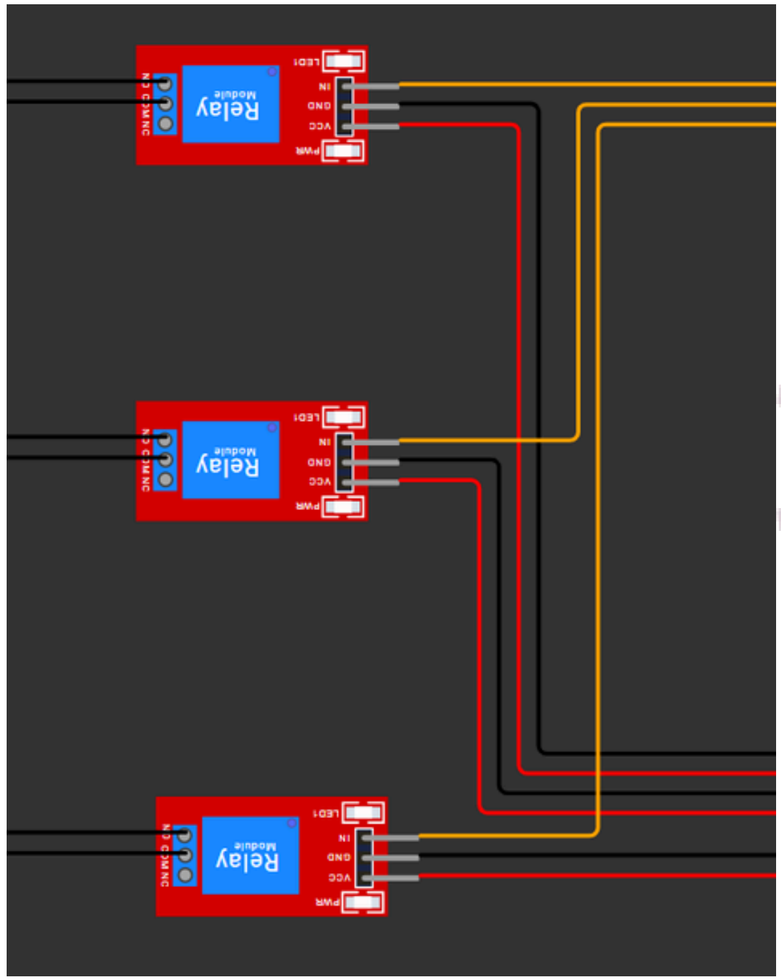
BLOCK DIAGRAM

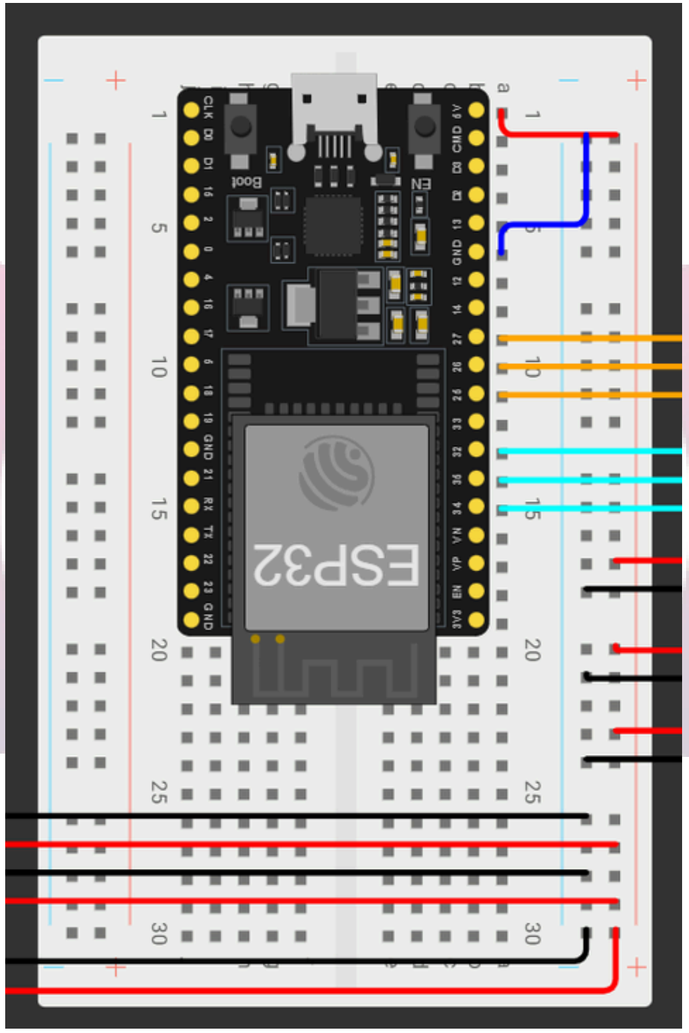


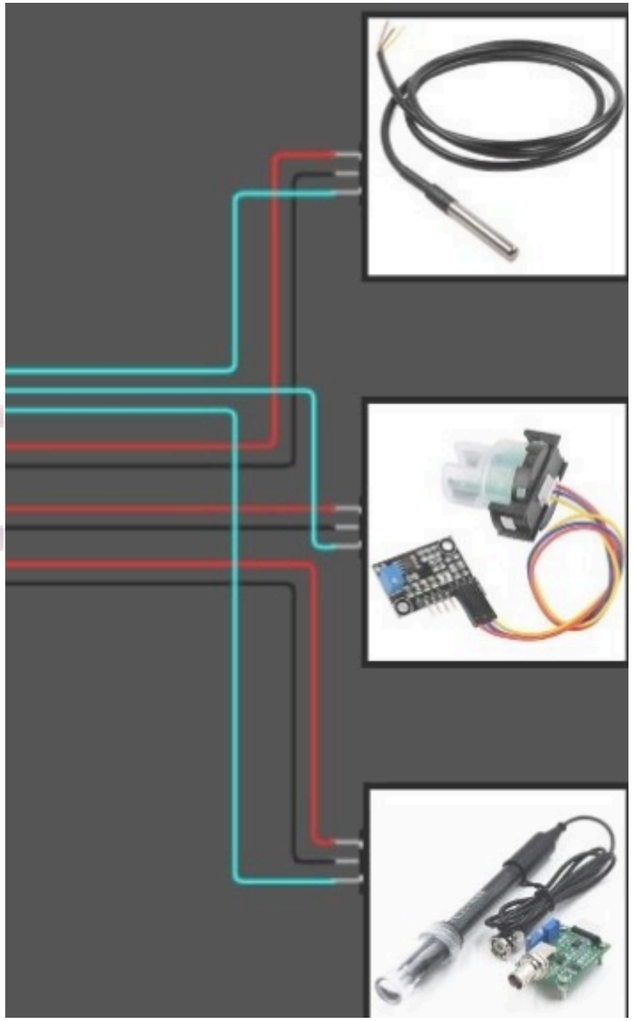
SCHEMATIC DIAGRAM



SCHEMATIC DIAGRAM

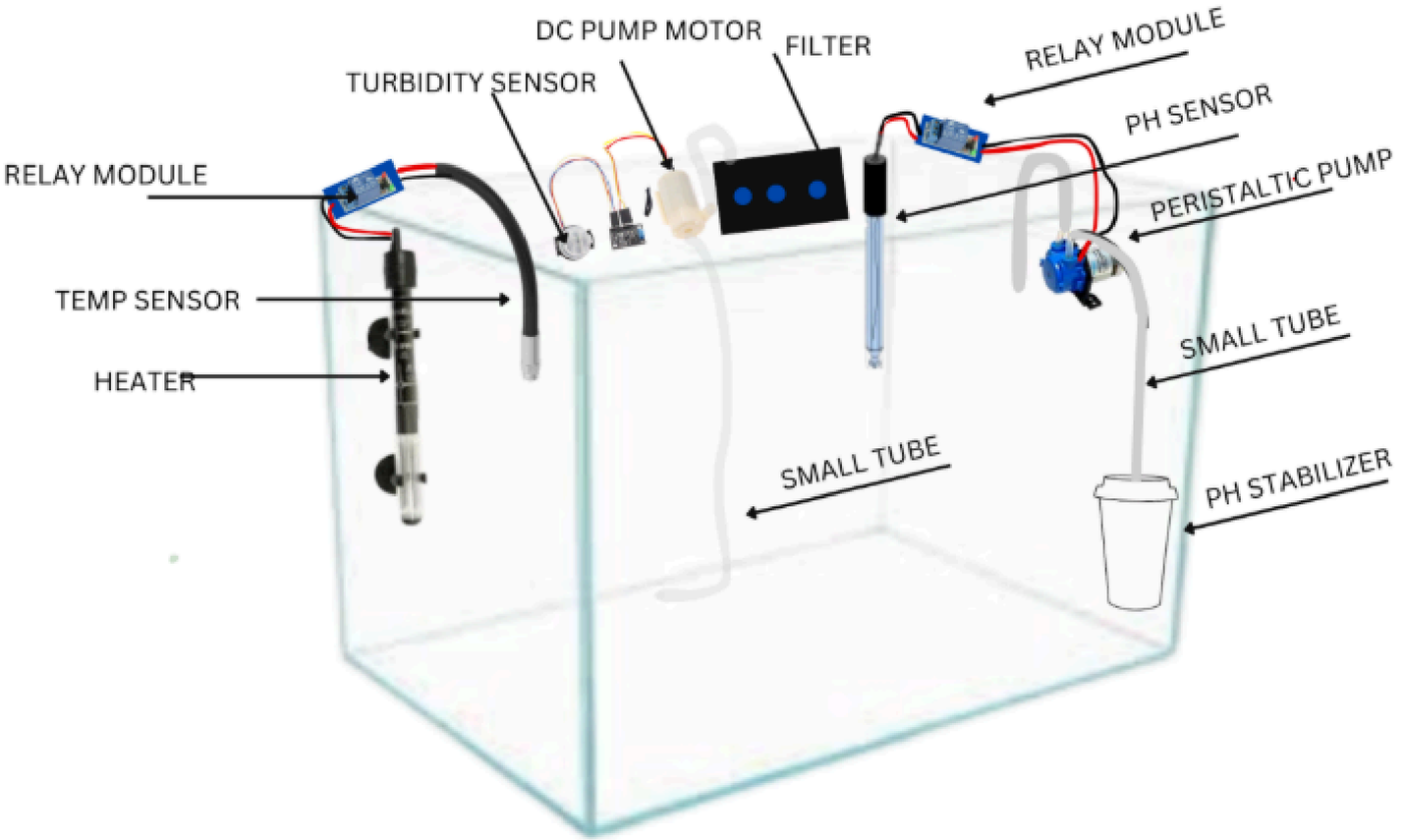






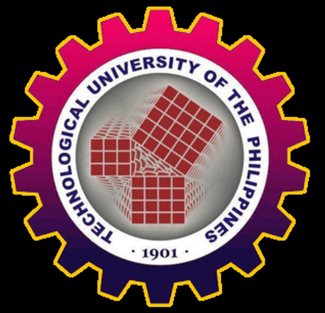
EXPECTED OUPUT

DIGITALIZED DRAWING



CODES

SENSORS




```
#include <OneWire.h>
#include <DallasTemperature.h>

// RELAYS
#define RELAY_HEATER 27      // Heater relay
#define RELAY_PUMP 25        // Peristaltic pump relay
#define RELAY_TURBIDITY 26   // Turbidity relay

// SENSORS
#define PH_SENSOR_PIN 34        // pH sensor analog pin
#define TURBIDITY_SENSOR_PIN 35 // Turbidity sensor analog pin
#define TEMP_SENSOR_PIN 32      // Temperature Sensor

// FIXED VALUES
#define TEMP_THRESHOLD 35.0
#define PH_HIGH 7.5  // Upper pH limit
#define PH_LOW 6.5   // Lower pH limit
#define TURBIDITY_THRESHOLD 100 // Adjust based on calibration

OneWire oneWire(TEMP_SENSOR_PIN);
DallasTemperature sensors(&oneWire);

void setup() {
    Serial.begin(115200);
    sensors.begin();

    pinMode(RELAY_HEATER, OUTPUT);
    pinMode(RELAY_PUMP, OUTPUT);
    pinMode(RELAY_TURBIDITY, OUTPUT);

    digitalWrite(RELAY_HEATER, HIGH); // Relays off initially (active-
    digitalWrite(RELAY_PUMP, HIGH);
    digitalWrite(RELAY_TURBIDITY, HIGH);
```

```
void loop() {
    // *Temperature Control*
    sensors.requestTemperatures();
    float temperature = sensors.getTempCByIndex(0);

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    if (temperature < TEMP_THRESHOLD) {
        digitalWrite(RELAY_HEATER, LOW);
        Serial.println("\xF0\x9F\x94\xA5 Heater ON");
    } else {
        digitalWrite(RELAY_HEATER, HIGH);
        Serial.println("\xE2\x9D\x84\xEF\xB8\x8F Heater OFF");
    }

    // *pH Monitoring & Display*
    int phValue = analogRead(PH_SENSOR_PIN);
    float voltage = (phValue / 4095.0) * 3.3; // Convert ADC to vc
    float pH = voltage * 3.5; // Adjust based on calibration

    Serial.print("\xF0\x9F\x93\xA2 Current pH Level: ");
    Serial.println(pH);

    // *pH Control: Peristaltic Pump Activation*
    bool pumpActivated = false;
    if (pH < PH_LOW) {
        Serial.println("⚠️ pH too LOW! Pumping base...");
        digitalWrite(RELAY_PUMP, LOW);
        pumpActivated = true;
    }
    else if (pH > PH_HIGH) {
        Serial.println("⚠️ pH too HIGH! Pumping acid...");
        digitalWrite(RELAY_PUMP, LOW);
```

```
    // *pH Control: Peristaltic Pump Activation*
    bool pumpActivated = false;
    if (pH < PH_LOW) {
        Serial.println("⚠️ pH too LOW! Pumping base...");
        digitalWrite(RELAY_PUMP, LOW);
        pumpActivated = true;
    }
    else if (pH > PH_HIGH) {
        Serial.println("⚠️ pH too HIGH! Pumping acid...");
        digitalWrite(RELAY_PUMP, LOW);
        pumpActivated = true;
    }
    else {
        Serial.println("✅ pH is within normal range. Pump OFF.");
        digitalWrite(RELAY_PUMP, HIGH);  // Ensure the pump is OFF when pH is normal
    }


    // *Turbidity Monitoring & Control*
    int turbidityValue = analogRead(TURBIDITY_SENSOR_PIN);
    Serial.print("🌊 Turbidity Value: ");
    Serial.println(turbidityValue);

    if (turbidityValue > TURBIDITY_THRESHOLD) {
        Serial.println("⚠️ Water is turbid or pH is out of range! Activating filter...");
        digitalWrite(RELAY_TURBIDITY, LOW);
    } else {
        Serial.println("✅ Water is clear. Filter OFF.");
        digitalWrite(RELAY_TURBIDITY, HIGH);
    }

    Serial.println("---------------------------------"); // Separator for readability
    delay(2000);
}
```

CODES

CONTROLLED WITH A DEVICE

```
// *BLYNK CONFIGURATION* (Must be defined before including Blynk libraries)
#define BLYNK_TEMPLATE_ID "TMPL6rscT-TIn"
#define BLYNK_TEMPLATE_NAME "SYSTEM AYOS"
#define BLYNK_AUTH_TOKEN "JBWfbF2Ns0NV1T12P8ZJ0MANjp7RFNCR"

// *Libraries*
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

// *WiFi Credentials*
char ssid[] = "HUAWEI-2.4G-S78n";
char pass[] = "cfShh8nq";

// *Relay Pins*
#define RELAY_HEATER 27
#define RELAY_PUMP 25
#define RELAY_TURBIDITY 26

// *Sensor Pins*
#define PH_SENSOR_PIN 34
#define TURBIDITY_SENSOR_PIN 35
#define TEMP_SENSOR_PIN 32

// *Thresholds & Calibration*
#define TEMP_THRESHOLD 26.0
#define PH_HIGH 9
#define TURBIDITY_THRESHOLD 120

// *pH Sensor Calibration*
float calibration_value = 21.34 - 0.7;
```

```
OneWire oneWire(TEMP_SENSOR_PIN);
DallasTemperature sensors(&oneWire);

float pH = 0.0;

void setup() {
    Serial.begin(115200);
    sensors.begin();

    pinMode(RELAY_HEATER, OUTPUT);
    pinMode(RELAY_PUMP, OUTPUT);
    pinMode(RELAY_TURBIDITY, OUTPUT);

    digitalWrite(RELAY_HEATER, HIGH);
    digitalWrite(RELAY_PUMP, HIGH);
    digitalWrite(RELAY_TURBIDITY, HIGH);

    // Connect to Blynk
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

void loop() {
    Blynk.run(); // Run Blynk

    // *Temperature Monitoring*
    sensors.requestTemperatures();
    float temperature = sensors.getTempCByIndex(0);

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    if (temperature < TEMP_THRESHOLD) {
        digitalWrite(RELAY_HEATER, LOW);
        Serial.println(" Heater ON");
```

CODES

CONTROLLED WITH A DEVICE

```
70      } else {
71          digitalWrite(RELAY_HEATER, HIGH);
72          Serial.println(" Heater OFF");
73      }
74
75      // *pH Monitoring & Display*
76      int phValue = analogRead(PH_SENSOR_PIN);
77      float voltage = (phValue / 4095.0) * 3.3;
78      pH = -5.70 * voltage + calibration_value;
79
80      Serial.print(" Current pH Level: ");
81      Serial.println(pH);
82
83      // *pH Control: Pump Activation*
84      if (pH > PH_HIGH) {
85          Serial.println(" pH too HIGH! Pumping acid...");
86          digitalWrite(RELAY_PUMP, LOW);
87      } else {
88          Serial.println(" pH is within normal range. Pump OFF.");
89          digitalWrite(RELAY_PUMP, HIGH);
90      }
91
92      // *Turbidity Monitoring & Control*
93      int read_ADC = analogRead(TURBIDITY_SENSOR_PIN);
94
95      // Adjusted for better sensitivity (max ADC ~ 2500)
96      if (read_ADC > 2500) read_ADC = 2500;
97
98      // Map ADC values to NTU range (higher resolution)
99      int ntu = map(read_ADC, 0, 2500, 300, 0);
100
101     Serial.print("🌊 Turbidity ADC: ");
102     Serial.print(read_ADC);
103     Serial.print(" | NTU: ");
104     Serial.println(ntu);
```

```
106     // *Determine water turbidity levels*
107     if (ntu < TURBIDITY_THRESHOLD ) {
108         Serial.println("✅ Water is Clean. Filter OFF.");
109         digitalWrite(RELAY_TURBIDITY, HIGH);
110     } else {
111         Serial.println("⚠️ Water is Turbid! Activating Filter...");
112         digitalWrite(RELAY_TURBIDITY, LOW);
113     }
114
115     Serial.println("---------------------------------");
116
117     // *Send Data to Blynk*
118     Blynk.virtualWrite(V1, pH);
119     Blynk.virtualWrite(V2, temperature);
120     Blynk.virtualWrite(V3, ntu);
121     Blynk.virtualWrite(V4, temperature < TEMP_THRESHOLD ? "Heater ON" : "Heater OFF");
122     Blynk.virtualWrite(V5, pH > PH_HIGH ? "Pumping Acid" : "Pump OFF");
123     Blynk.virtualWrite(V6, ntu > TURBIDITY_THRESHOLD ? "Filter ON" : "Filter OFF");
124
125     delay(2000);
126 }
```

CODES

AUTOMATIC

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <SimpleTimer.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// *RELAY PINS*
#define RELAY_HEATER 27
#define RELAY_PUMP 25
#define RELAY_TURBIDITY 26

// *SENSOR PINS*
#define PH_SENSOR_PIN 34
#define TURBIDITY_SENSOR_PIN 35
#define TEMP_SENSOR_PIN 32

// *FIXED VALUES*
#define TEMP_THRESHOLD 35.0
#define PH_HIGH 7.5
#define PH_LOW 6.5
#define TURBIDITY_THRESHOLD 100

// *pH Sensor Calibration*
float calibration_value = 21.34 - 0.7;

// *Display Settings*
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

SimpleTimer timer;
OneWire oneWire(TEMP_SENSOR_PIN);
DallasTemperature sensors(&oneWire);
```

```
void setup() {
    Serial.begin(115200);
    sensors.begin();

    pinMode(RELAY_HEATER, OUTPUT);
    pinMode(RELAY_PUMP, OUTPUT);
    pinMode(RELAY_TURBIDITY, OUTPUT);

    digitalWrite(RELAY_HEATER, HIGH);
    digitalWrite(RELAY_PUMP, HIGH);
    digitalWrite(RELAY_TURBIDITY, HIGH);

    // *Initialize OLED Display*
    Wire.begin();
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("SSD1306 allocation failed");
        while (true);
    }
    display.clearDisplay();
    display.setTextColor(WHITE);

    timer.setInterval(500L, display_pHValue);

    Serial.println("System Initialized.");
}

void loop() {
    timer.run();

    // *Temperature Monitoring*
    sensors.requestTemperatures();
    float temperature = sensors.getTempCByIndex(0);
```

CODES

AUTOMATIC

```
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    if (temperature < TEMP_THRESHOLD) {
        digitalWrite(RELAY_HEATER, LOW);
        Serial.println("🔥 Heater ON");
    } else {
        digitalWrite(RELAY_HEATER, HIGH);
        Serial.println("❄️ Heater OFF");
    }

    // *pH Monitoring & Display*
    int phValue = analogRead(PH_SENSOR_PIN);
    float voltage = (phValue / 4095.0) * 3.3;  // ESP32 uses 3.3V ADC
    float pH = -5.70 * voltage + calibration_value;

    Serial.print("🧪 Current pH Level: ");
    Serial.println(pH);

    // *pH Control: Pump Activation*
    if (pH < PH_LOW) {
        Serial.println("⚠️ pH too LOW! Pumping base...");
        digitalWrite(RELAY_PUMP, LOW);
    }
    else if (pH > PH_HIGH) {
        Serial.println("⚠️ pH too HIGH! Pumping acid...");
        digitalWrite(RELAY_PUMP, LOW);
    }
    else {
        Serial.println("✅ pH is within normal range. Pump OFF.");
        digitalWrite(RELAY_PUMP, HIGH);
    }
```

```
    // *Turbidity Monitoring & Control*
    int turbidityValue = analogRead(TURBIDITY_SENSOR_PIN);
    Serial.print("🌊 Turbidity Value: ");
    Serial.println(turbidityValue);

    if (turbidityValue > TURBIDITY_THRESHOLD) {
        Serial.println("⚠️ Water is turbid! Activating filter...");
        digitalWrite(RELAY_TURBIDITY, LOW);
    } else {
        Serial.println("✅ Water is clear. Filter OFF.");
        digitalWrite(RELAY_TURBIDITY, HIGH);
    }

    Serial.println("---------------------------------");
    delay(2000);
}

// *Function to Display pH on OLED*
void display_pHValue() {
    display.clearDisplay();
    display.setTextSize(2);
    display.setCursor(0, 0);
    display.print("pH:");

    display.setTextSize(2);
    display.setCursor(55, 0);
    display.print(pH);

    display.display();
}
```

CODES

```ino
// *BLYNK CONFIGURATION*
#define BLYNK_TEMPLATE_ID "TMPL6rscT-TIn"
#define BLYNK_TEMPLATE_NAME "SYSTEM AYOS"
#define BLYNK_AUTH_TOKEN "JBWfbF2Ns0NV1T12P8ZJ0MANjp7RFNCR"

// *Libraries*
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

// *WiFi Credentials*
char ssid[] = "Redmi Note 13 Pro 5G";
char pass[] = "jettaxel";

// *Relay Pins*
#define RELAY_HEATER 27
#define RELAY_PUMP 25
#define RELAY_TURBIDITY 26

// *Sensor Pins*
#define PH_SENSOR_PIN 34
#define TURBIDITY_SENSOR_PIN 35
#define TEMP_SENSOR_PIN 32

// *pH Sensor Calibration*
float calibration_value = 21.34 - 0.7;

OneWire oneWire(TEMP_SENSOR_PIN);
DallasTemperature sensors(&oneWire);

float pH = 0.0;
```

```ino
void setup() {
    Serial.begin(115200);
    sensors.begin();

    pinMode(RELAY_HEATER, OUTPUT);
    pinMode(RELAY_PUMP, OUTPUT);
    pinMode(RELAY_TURBIDITY, OUTPUT);

    digitalWrite(RELAY_HEATER, HIGH);
    digitalWrite(RELAY_PUMP, HIGH);
    digitalWrite(RELAY_TURBIDITY, HIGH);

    // Connect to Blynk
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

void loop() {
    Blynk.run();

    // *Temperature Monitoring*
    sensors.requestTemperatures();
    float temperature = sensors.getTempCByIndex(0);

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");

    // *pH Monitoring*
    int phValue = analogRead(PH_SENSOR_PIN);
    float voltage = (phValue / 4095.0) * 3.3;
    pH = -5.70 * voltage + calibration_value;

    Serial.print("Current pH Level: ");
    Serial.println(pH);
```

CODES

```
    // *Turbidity Monitoring*
    int read_ADC = analogRead(TURBIDITY_SENSOR_PIN);
    if (read_ADC > 2500) read_ADC = 2500;
    int ntu = map(read_ADC, 0, 2500, 300, 0);

    Serial.print("Turbidity ADC: ");
    Serial.print(read_ADC);
    Serial.print(" | NTU: ");
    Serial.println(ntu);

    // *Send Sensor Data to Blynk*
    Blynk.virtualWrite(V1, pH);
    Blynk.virtualWrite(V2, temperature);
    Blynk.virtualWrite(V3, ntu);

    // *Update Relay Status in Serial Monitor & Blynk*
    Serial.print("Heater Status: ");
    Serial.println(digitalRead(RELAY_HEATER) == LOW ? "ON" : "OFF");
    Blynk.virtualWrite(V4, digitalRead(RELAY_HEATER) == LOW ? "Heater ON" : "Heater OFF");

    Serial.print("Pump Status: ");
    Serial.println(digitalRead(RELAY_PUMP) == LOW ? "ON" : "OFF");
    Blynk.virtualWrite(V5, digitalRead(RELAY_PUMP) == LOW ? "Pump ON" : "Pump OFF");

    Serial.print("Turbidity Filter Status: ");
    Serial.println(digitalRead(RELAY_TURBIDITY) == LOW ? "ON" : "OFF");
    Blynk.virtualWrite(V6, digitalRead(RELAY_TURBIDITY) == LOW ? "Filter ON" : "Filter OFF");

    Serial.println("---------------------------------");

    delay(2000);
}
```

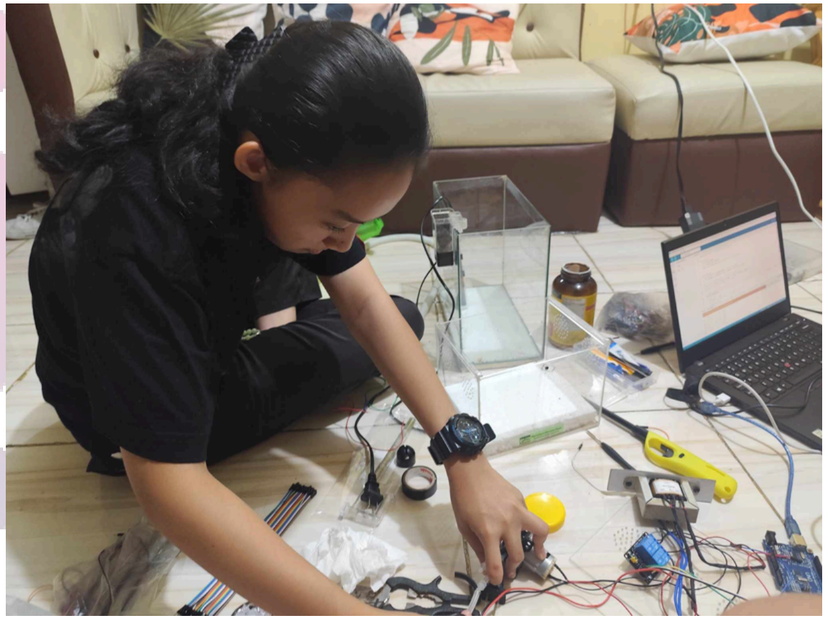
```
BLYNK_WRITE(V8) {
    int value = param.asInt();
    digitalWrite(RELAY_PUMP, value ? LOW : HIGH);
    Serial.print("Manual Pump Control: ");
    Serial.println(value ? "ON" : "OFF");
    Blynk.virtualWrite(V5, value ? "Pump ON" : "Pump OFF");
}

BLYNK_WRITE(V9) {
    int value = param.asInt();
    digitalWrite(RELAY_TURBIDITY, value ? LOW : HIGH);
    Serial.print("Manual Turbidity Filter Control: ");
    Serial.println(value ? "ON" : "OFF");
    Blynk.virtualWrite(V6, value ? "Filter ON" : "Filter OFF");
}
```

DOCUMENTATION

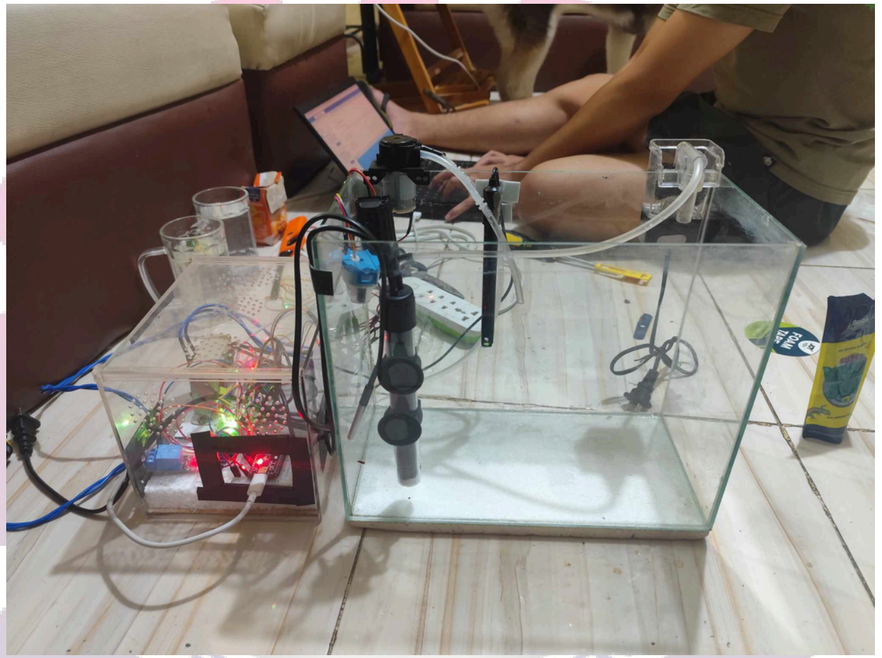


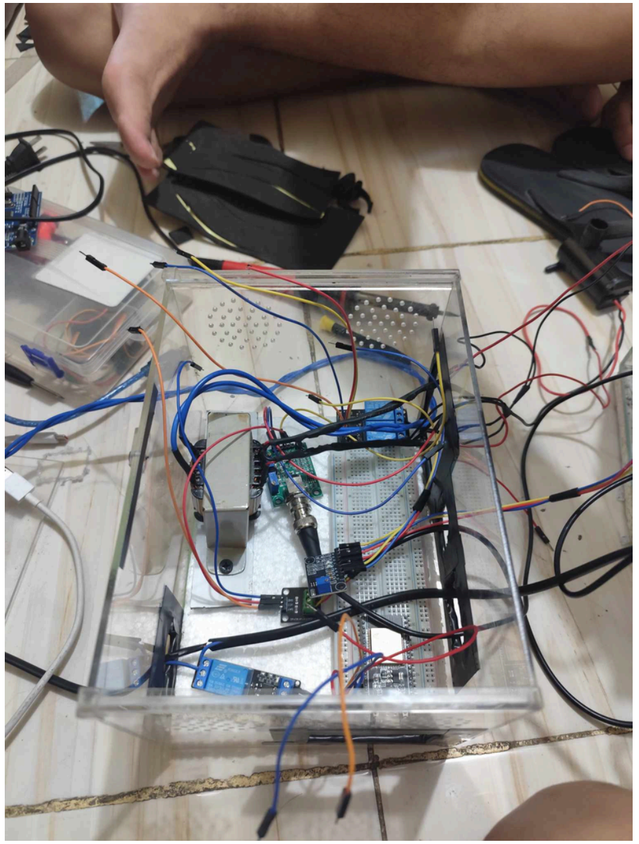




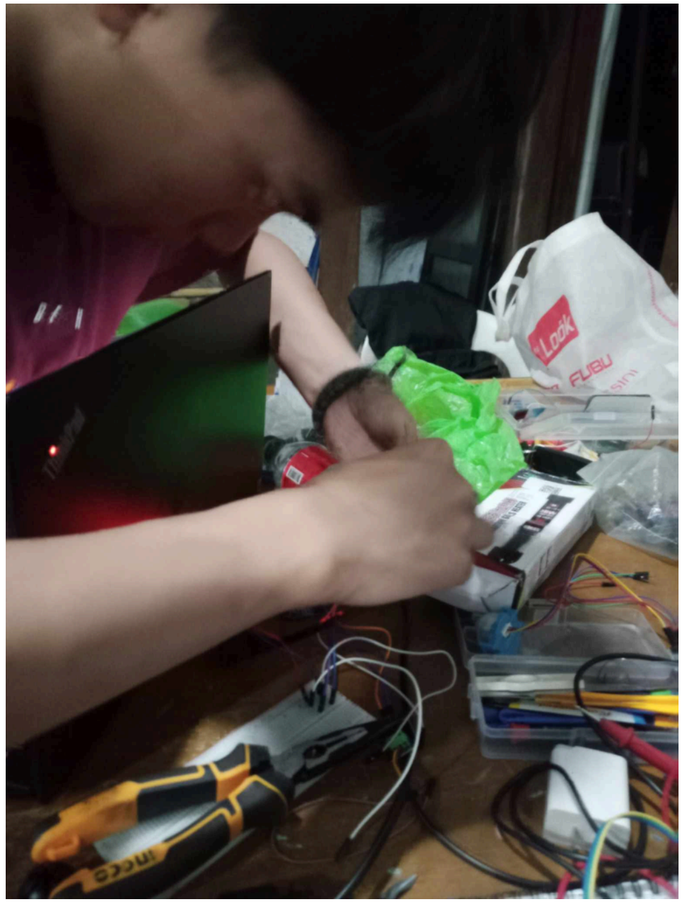
DOCUMENTATION



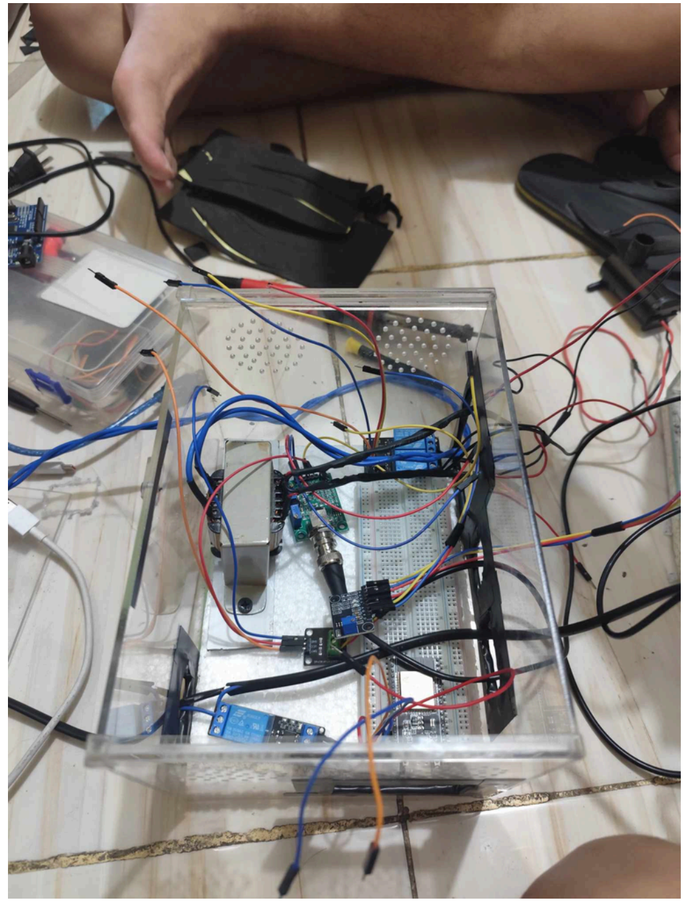




DOCUMENTATION







REFERENCES

- Controlling a Relay using Temperature (Temperature Controlled Relay with Thermocouple, Arduino, LCD)
  https://www.youtube.com/watch?v=GW4YsLnrO2s

- How to Make an Amazing DIY Top Filter for Your Aquarium at Home!
  https://www.youtube.com/watch?v=p17p1JHbETU

- pH Sensor and Peristaltic Pumps
  https://www.youtube.com/watch?v=656NEt114j0

- How to use & Calibration pH Meter DFRobot Water quality testing With Arduino
  https://www.youtube.com/watch?v=dS9-BYoazUw